Министерство науки и высшего образования РФ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Уфимский университет науки и технологий»

Кафедра: Высокопроизводительных вычислений и дифференциальных уравнений

МОДЕЛИРОВАНИЕ ДВУМЕРНЫХ ДИФФУЗИОННЫХ ПРОЦЕССОВ МЕТОДОМ НЕПРЕРЫВНЫХ СЛУЧАЙНЫХ БЛУЖДАНИЙ

ОТЧЕТ

к лабораторной работе №4 по дисциплине

«Математическое моделирование»

Группа МКН-418Б	Фамилия И. О.	Подпись	Дата	Оценка
Студент	Халитова А.А.			
Принял	Феоктистов Б.А.			

Уфа 2026

Содержание

1 Теоретическая часть **4**

2 Практическая часть **5**

2.1 Построение обратной функции 5

2.2 Моделирование движения частиц 5

2.3 Рассчитанные параметры 11

2.4 Построение аппроксимации 13

3 Вывод **16**

4 Листинг **17**

Цель работы: получить навык статистического моделирования диффузионных процессов методом случайных блужданий.

1 Теоретическая часть

Рассматривается случайный процесс движения диффундирующей частицы на плоскости. В начальный момент времени частица находится в начале координат. Затем частица осуществляет «прыжок» на случайную величину l, подчиняющуюся заданному закону распределения с плотностью вероятности $f(l)$. Направление прыжка выбирается произвольно из четырех возможных: влево, вправо, вверх или вниз. На следующем шаге процесс повторяется с новым случайным значением l. Частица должна выполнить N «прыжков». Весь процесс необходимо повторить для M частиц.

Вычислить функции распределения частиц $P_N(x)$, $P_N(y)$ по двум координатным направлениям, и рассчитать значения:

$$\langle x_N \rangle = \frac{1}{M}\sum_{i=1}^{M} x_i, \quad \langle y_N \rangle = \frac{1}{M}\sum_{i=1}^{M} y_i, \quad \langle R_N \rangle = \sqrt{\langle x_N \rangle^2 + \langle y_N \rangle^2},$$

где (x_i, y_i) - координата i-й частицы после N прыжков.

Также вычисляются параметры:

$$\langle x_N^2 \rangle = \frac{1}{M}\sum_{i=1}^{M} x_i^2, \quad \langle y_N^2 \rangle = \frac{1}{M}\sum_{i=1}^{M} y_i^2,$$

$$\langle \Delta x_N^2 \rangle = \langle x_N^2 \rangle - \langle x_N \rangle^2, \quad \langle \Delta y_N^2 \rangle = \langle y_N^2 \rangle - \langle y_N \rangle^2,$$

как функции N. После этого находится средний квадрат полного смещения частиц:

$$\langle \Delta R_N^2 \rangle = \langle \Delta x_N^2 \rangle + \langle \Delta y_N^2 \rangle.$$

Для больших значений N выполнить аппроксимацию этой величины степенной зависимостью

$$\langle \Delta R_N^2 \rangle \approx \mu N^{\nu}. \tag{1}$$

В зависимости от найденного значения показателя ν сделать вывод о близости асимптотического распределения к нормальному закону.

Все расчеты выполнить для значений функции плотности распределения вероятности длины прыжка, указанной в индивидуальном задании. При этом постоянная A находится из условия:

$$\int_0^{\infty} f(l)dl = 1. \tag{2}$$

Для построения генератора случайных чисел, подчиняющихся заданной плотности распределения $f(l)$, использовать формулу:

$$l = F^{-1}(r),$$

где r – равномерно распределенная на отрезке $[0, 1]$ случайная величина и

$$F(l) = \int_0^l f(x)dx.$$

2 Практическая часть

2.1 Построение обратной функции

Практическая часть решается согласно варианту №7.

Функция плотности распределения вероятности длины прыжка:

$$f(l) = \frac{Al^2}{l^6 + 1}. \tag{3}$$

Интеграл функции (3) равен:

$$F(l) = \frac{A}{3} \operatorname{arctg}(l^3).$$

Их выражения (2):

$$A = \frac{6}{\pi}.$$

Тогда обратная фунция:

$$l = F^{-1}(r) = \sqrt[3]{\operatorname{tg}(\frac{\pi r}{2})}. \tag{4}$$

2.2 Моделирование движения частиц

Рассмотрим систему из частиц численности $M \in 10^2$, 10^3, 10^4. Для каждой величины M смоделируем траектории на протяжении N шагов, где параметр N принимает значения:

$$N = 500, 1000, \ldots, 5000.$$

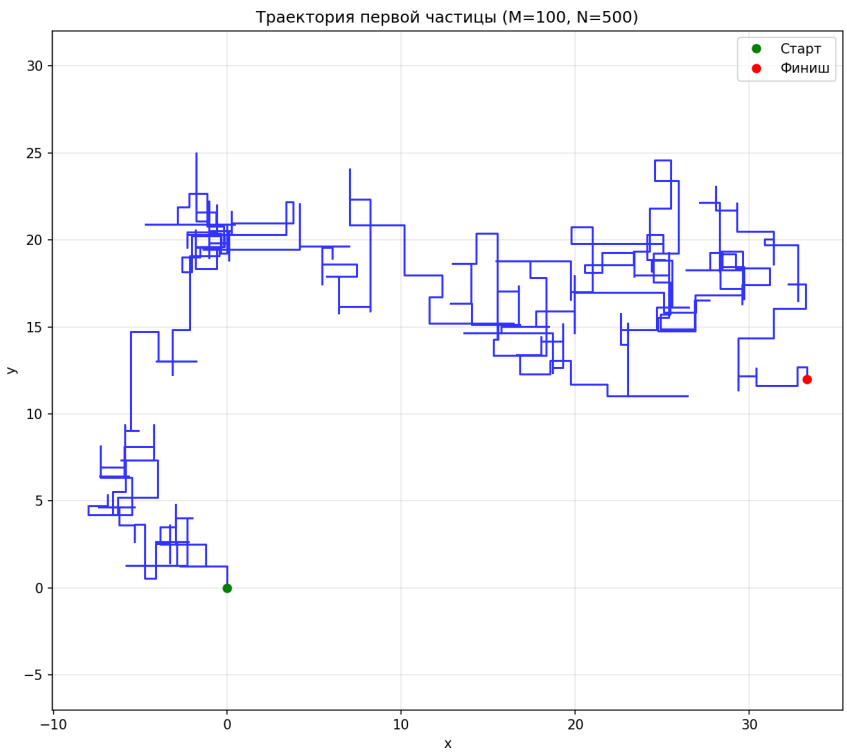

Рисунок 1 — Траектория движения первой точки M = 100, N = 500

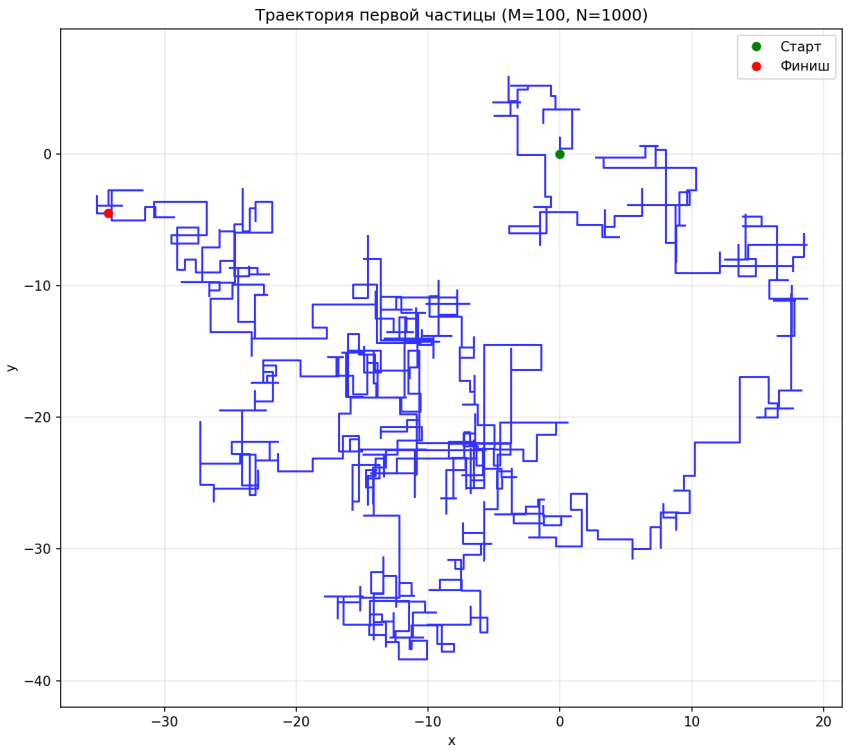

Рисунок 2 — Траектория движения первой точки M = 100, N = 1000

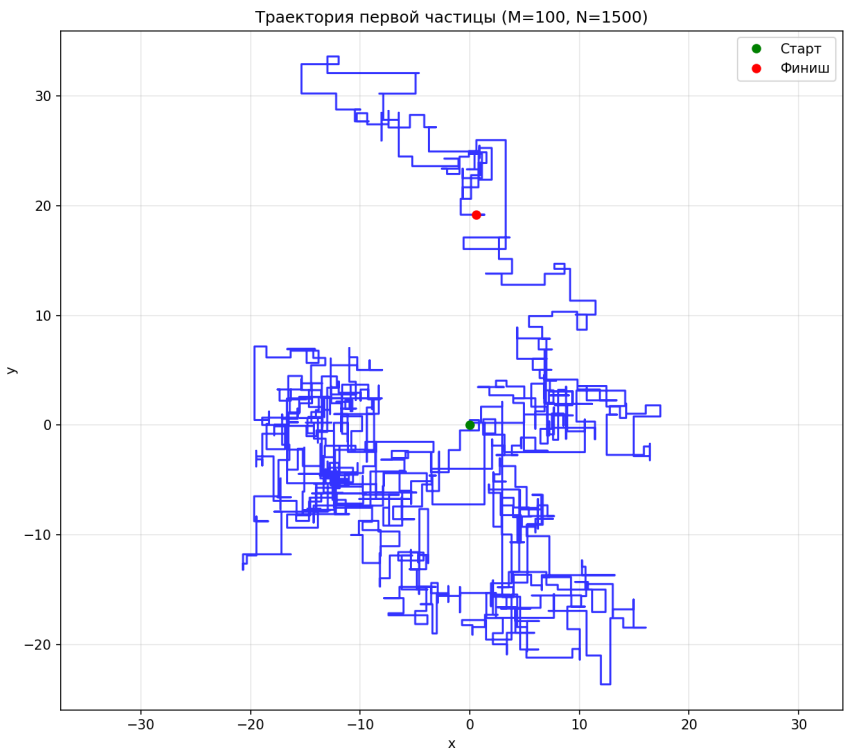

Рисунок 3 — Траектория движения первой точки M = 100, N = 1500

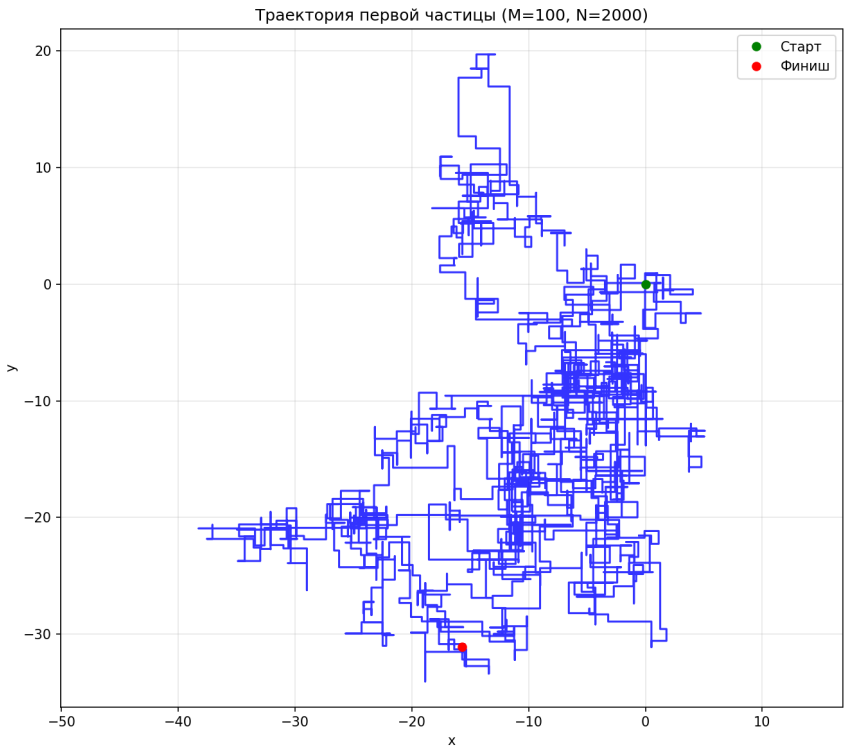

Рисунок 4 — Траектория движения первой точки M = 100, N = 2000

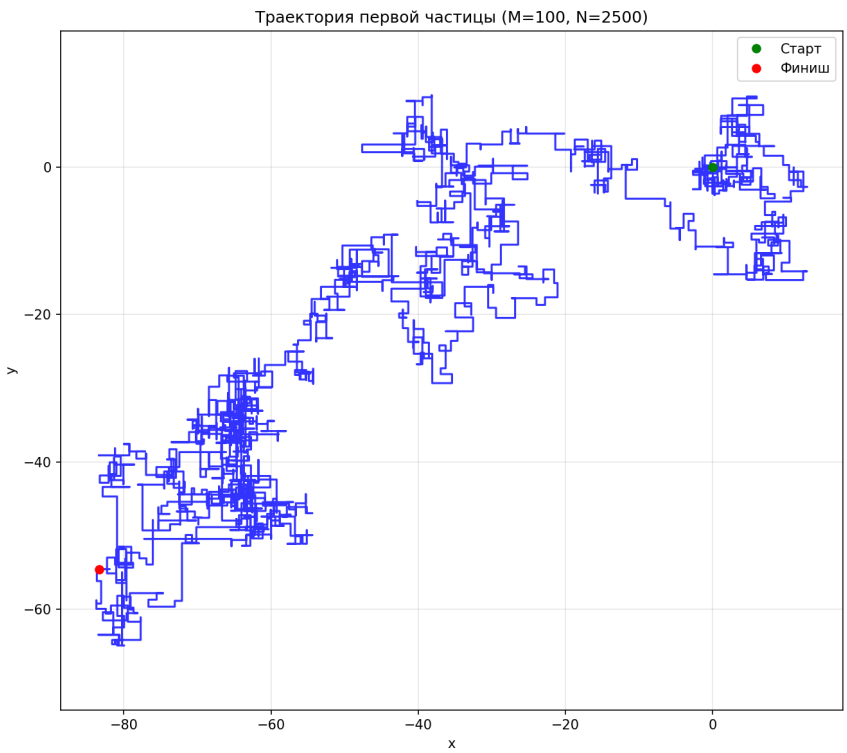

Рисунок 5 — Траектория движения первой точки M = 100, N = 2500

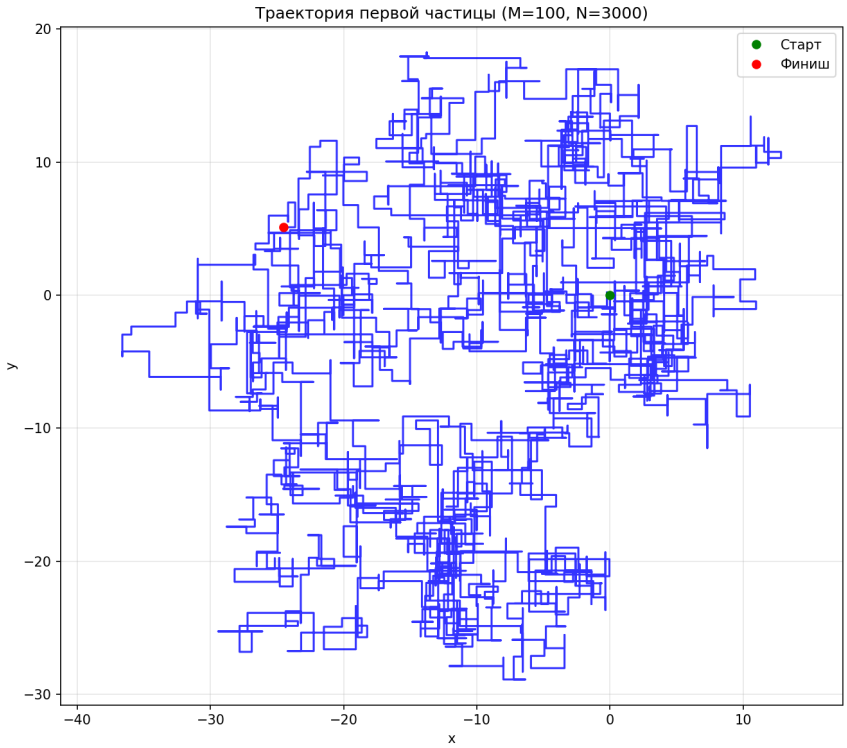

Рисунок 6 — Траектория движения первой точки M = 100, N = 3000

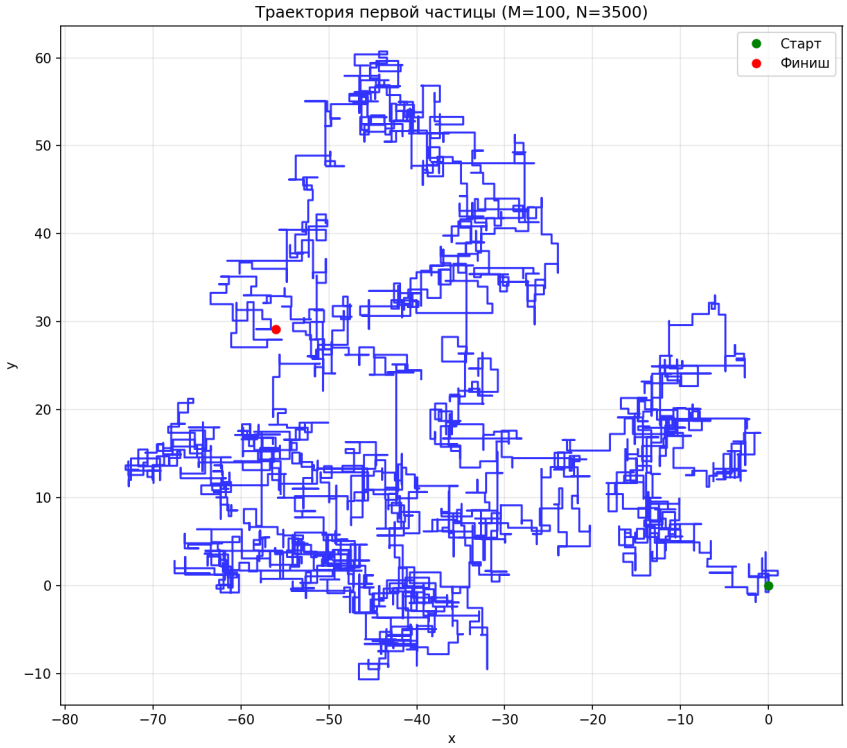

Рисунок 7 — Траектория движения первой точки M = 100, N = 3500

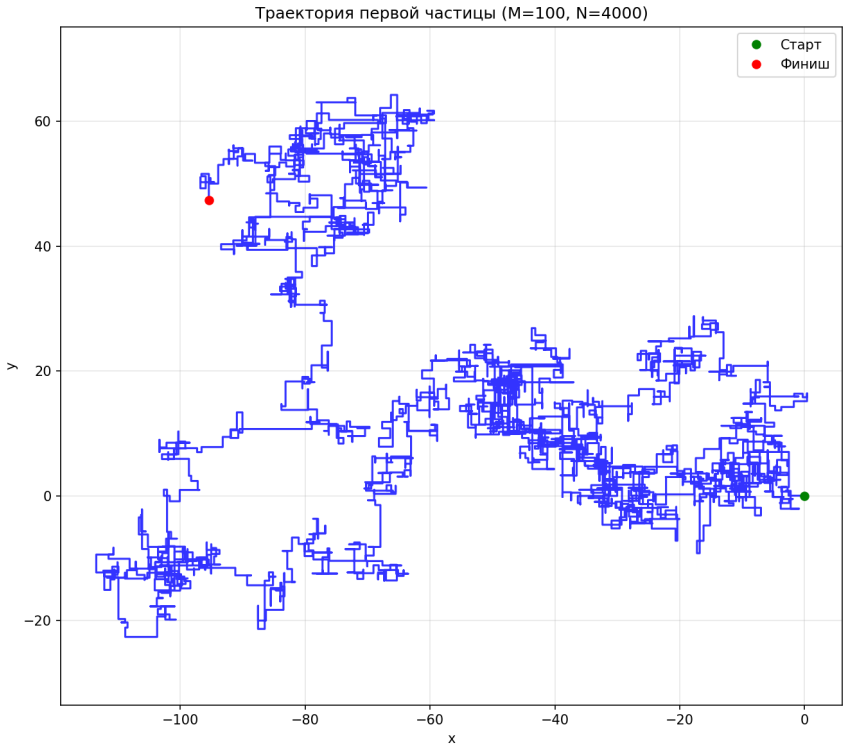

Рисунок 8 — Траектория движения первой точки M = 100, N = 4000

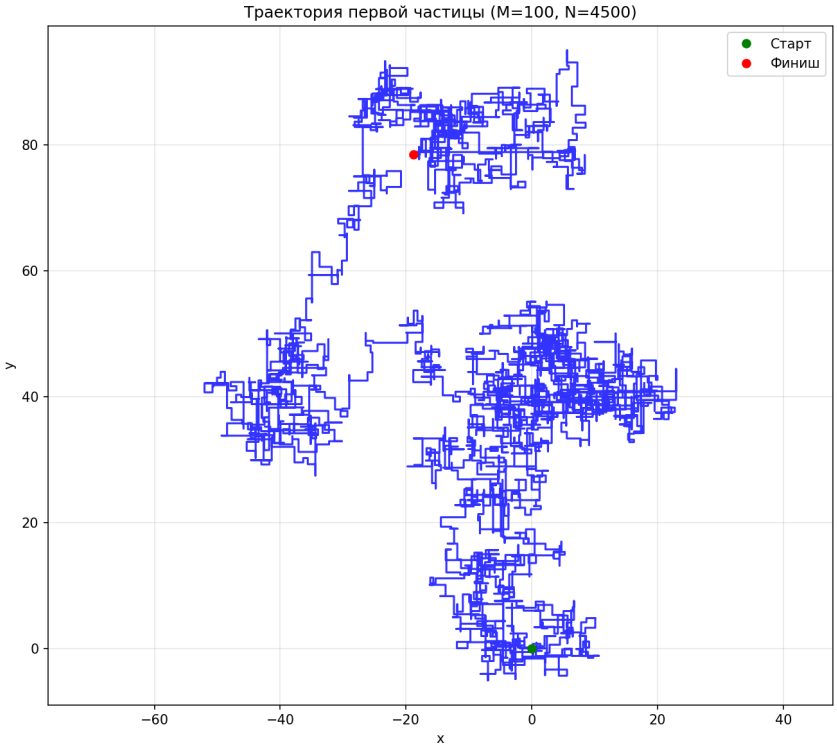

Рисунок 9 — Траектория движения первой точки M = 100, N = 4500

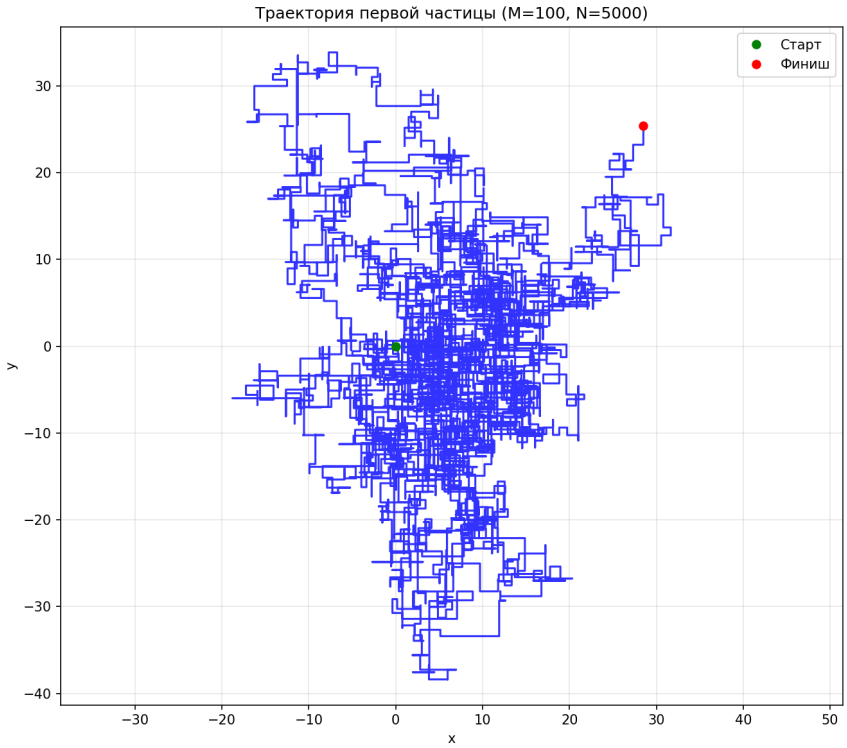

Рисунок 10 — Траектория движения первой точки M = 100, N = 5000

2.3 Рассчитанные параметры

Для каждого количества частиц рассчитаем значения мат ожидания и вариации.

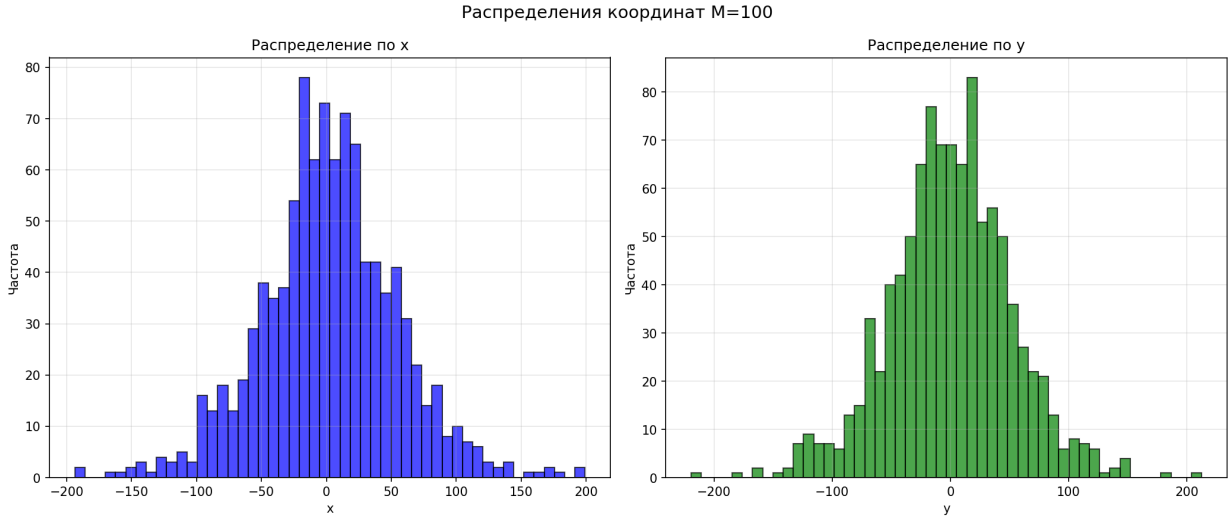

Рисунок 11 — Гистограмма распределения M = 100

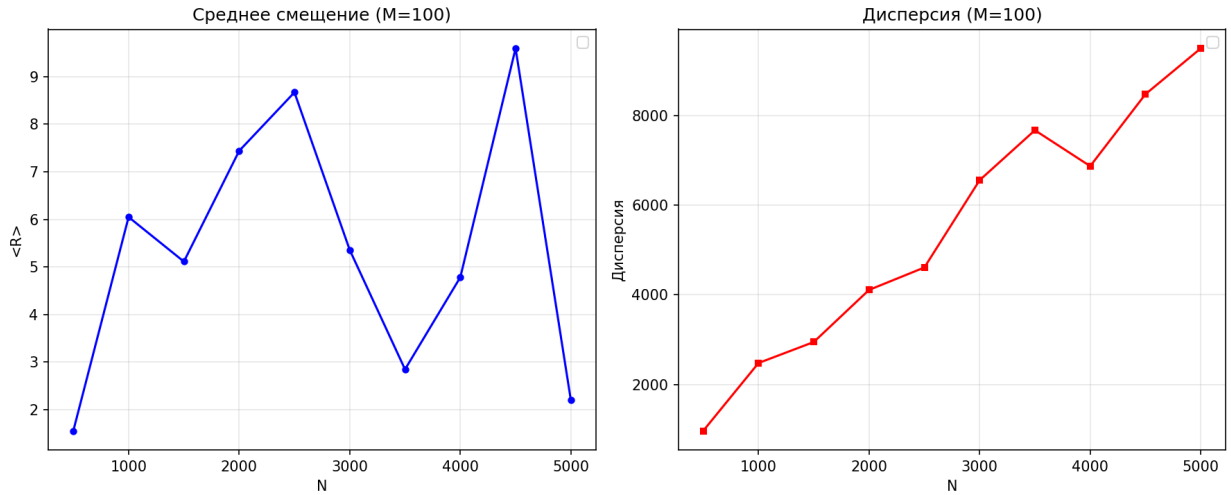

Рисунок 12 — Статистика для распределения M = 100

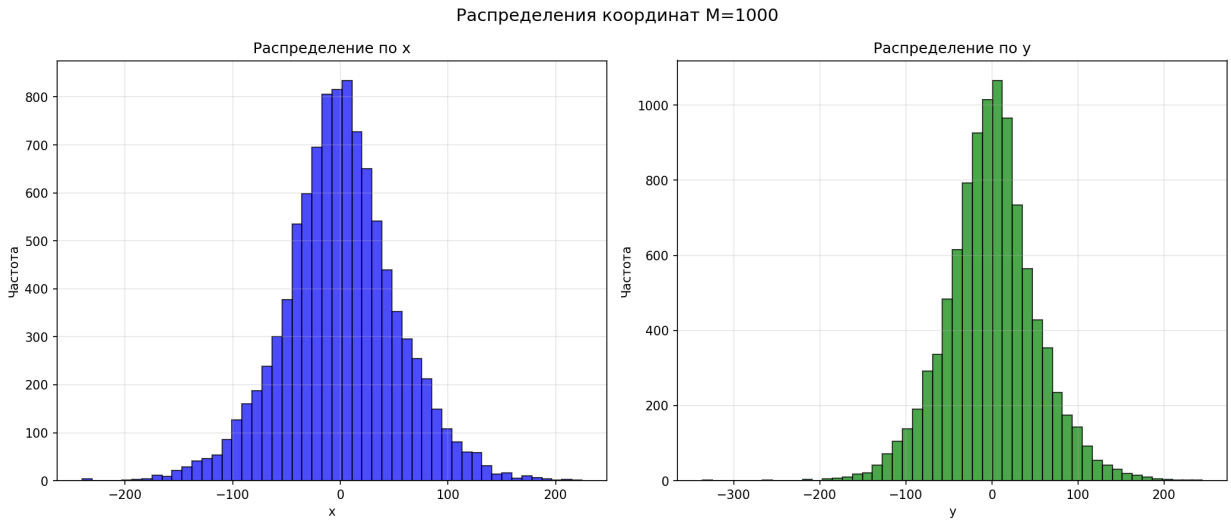

Рисунок 13 — Гистограмма распределения M = 1000

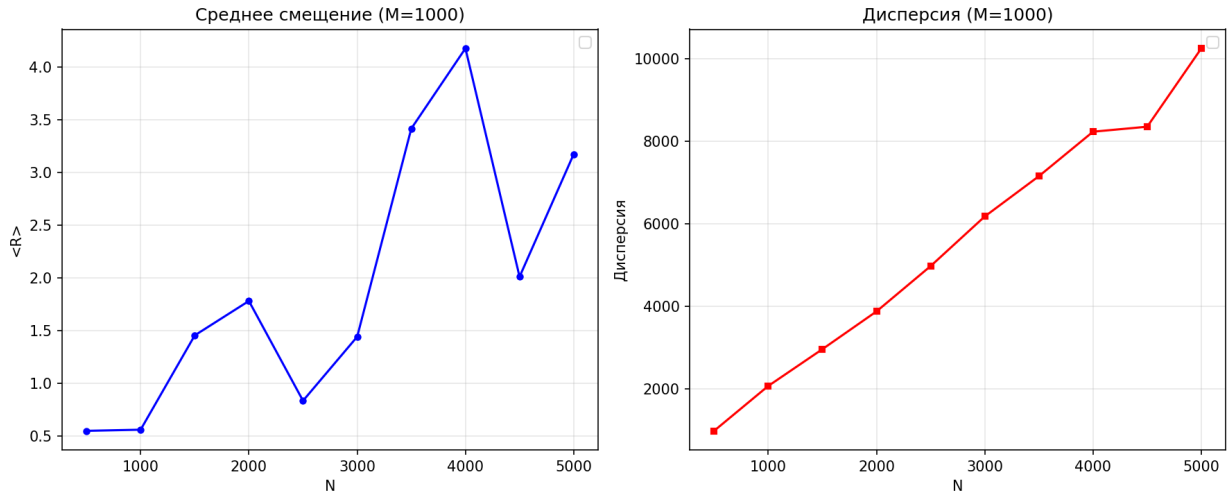

Рисунок 14 — Статистика для распределения M = 1000

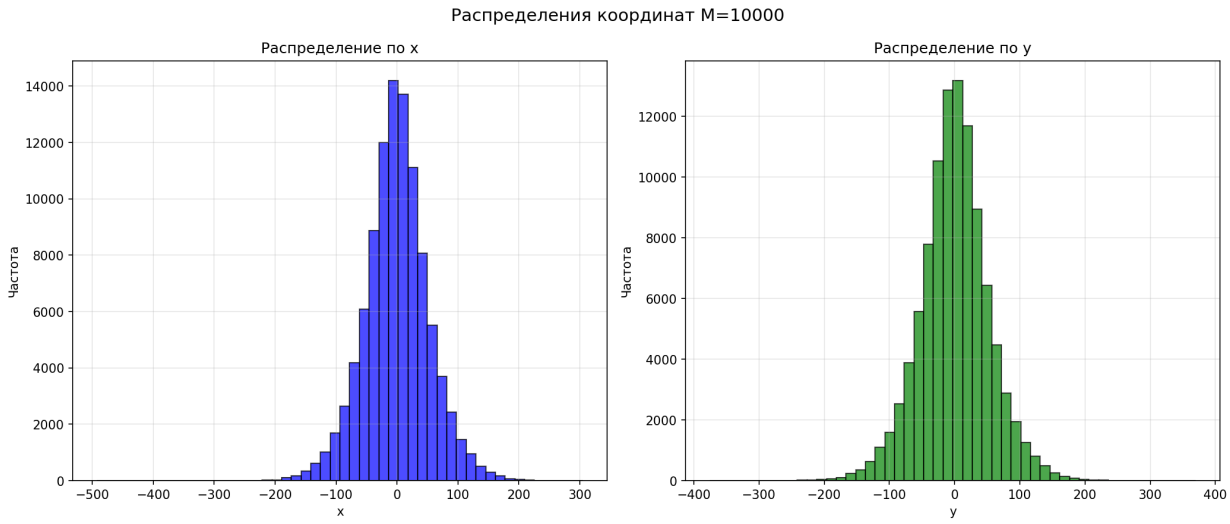

Рисунок 15 — Гистограмма распределения M = 10000

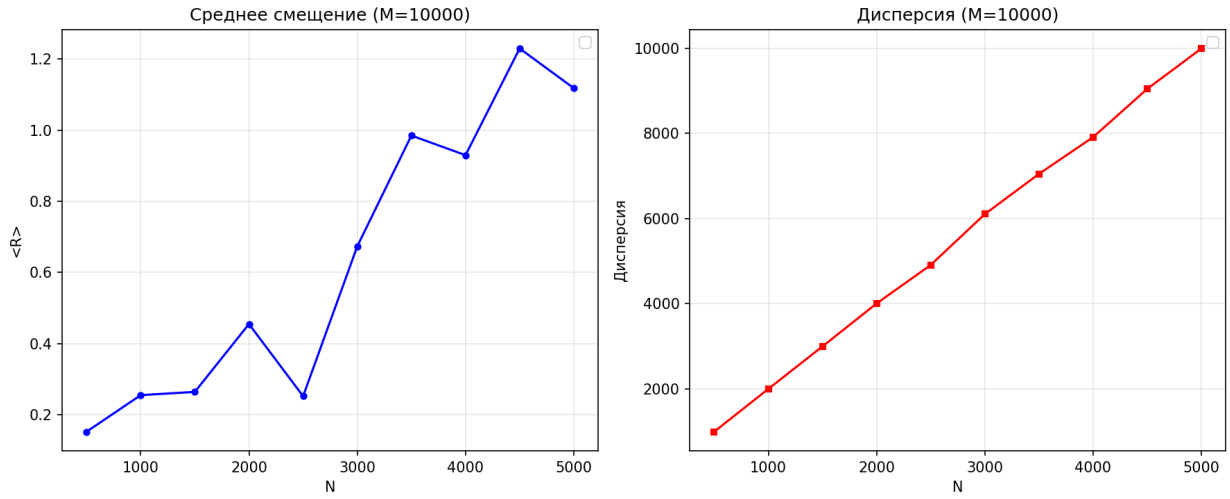

Рисунок 16 — Статистика для распределения M = 10000

2.4 Построение аппроксимации

При помощи метода наименьших квадратов построены аппроксимации (1).

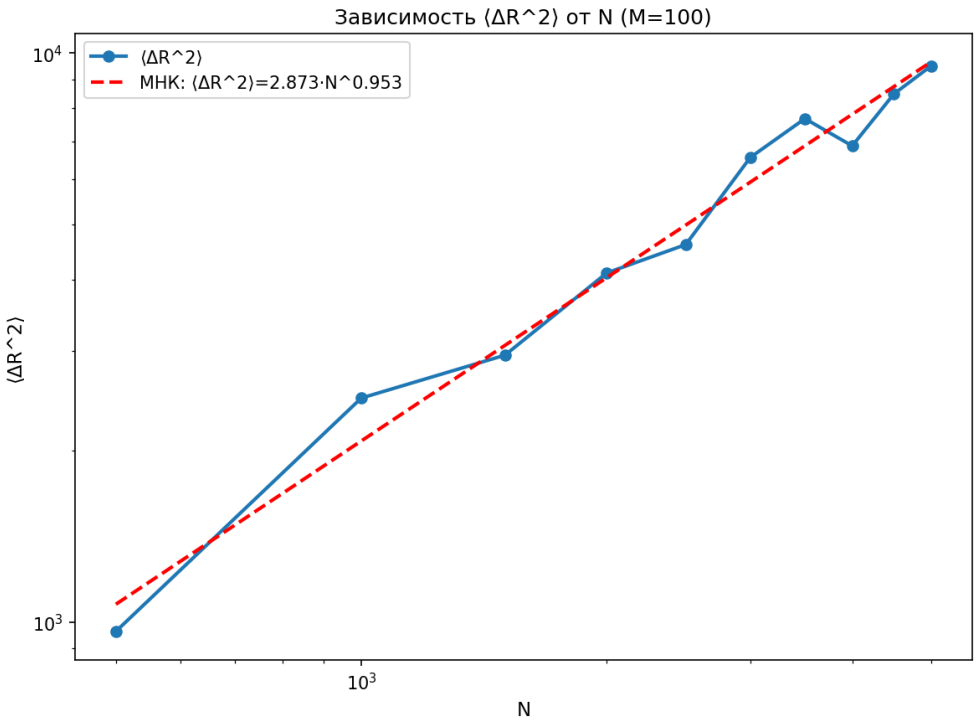

Рисунок 17 — Аппроксимация зависимости M = 100

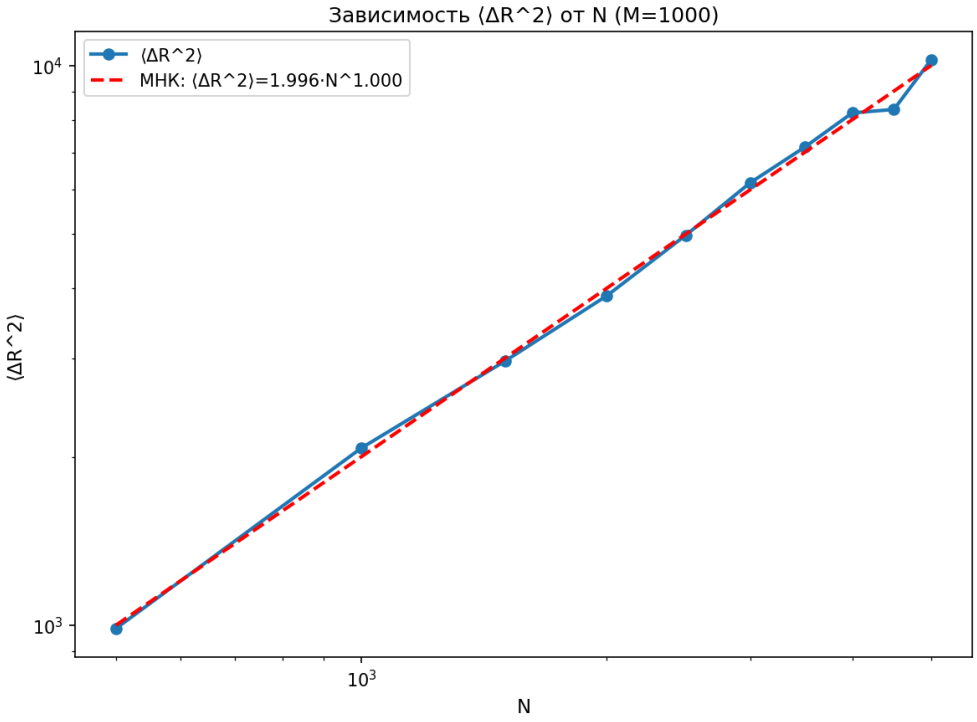

Рисунок 18 — Аппроксимация зависимости M = 1000

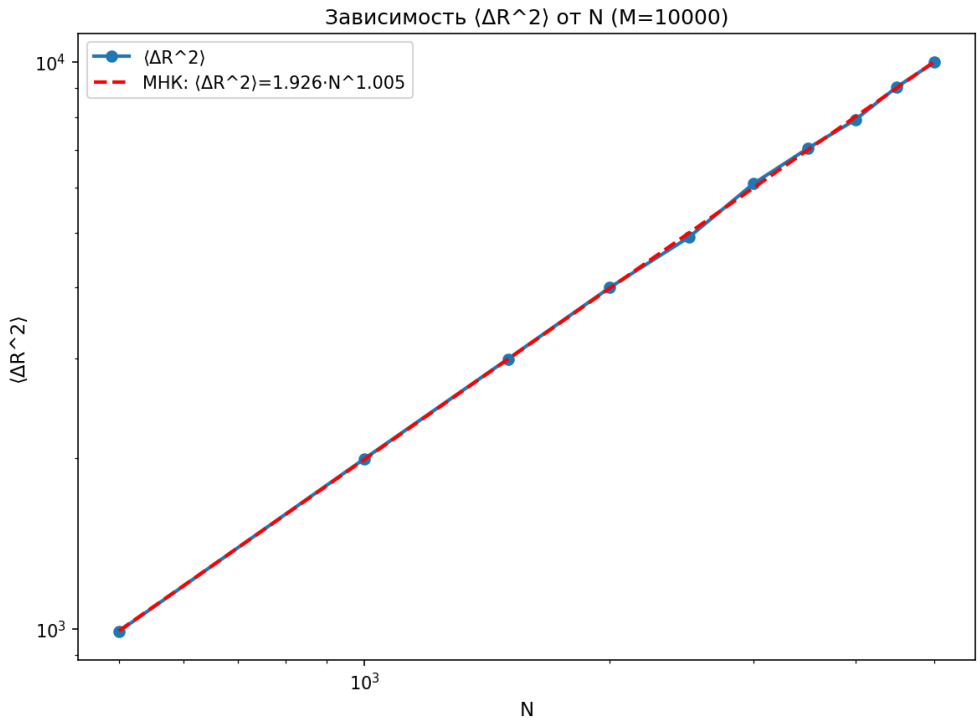

Рисунок 19 — Аппроксимация зависимости M = 10000

Полученные параметры аппроксимации представлены в таблице 1.

Таблица 1 — Параметры аппроксимации

M	A	ν
100	2.473	0.980
1000	1.497	1.039
10000	1.982	1.001

Рассчитаем теоретические моменты распределения плотности (3):

$$M_1 = \int_0^\infty l f(l) dl = 1.15,$$

$$M_2 = \int_0^\infty l^2 f(l) dl = 2.$$

Полученные значения ν близки к единице, что соответствует нормальной диффузии. Теоретические моменты распределения конечны, что также указывает на применимость центральной предельной теоремы и нормальный характер диффузии. Таким образом, асимптотическое распределение частиц стремится к нормальному закону.

3 Вывод

В ходе лабораторной работы была реализована программа, моделирующая диффузионные процессы методом случайных блужданий. Из экспериментов установили, что распределения подчиняются нормальному закону.

4 Листинг

main.cpp

```cpp
#include <omp.h>
#include <pybind11/embed.h>
#include <pybind11/numpy.h>
#include <pybind11/stl.h>

#include <chrono>
#include <cmath>
#include <iostream>
#include <random>
#include <vector>

namespace py = pybind11;

struct Statistics {
        double mean_x, mean_y;
        double var_x, var_y;
        double var_R, mean_R;
        std::vector<double> x_coords, y_coords;
        std::vector<std::vector<double>> trajectories_x;
        std::vector<std::vector<double>> trajectories_y;
};

class RandomWalksVisualizer {
        private:
        const double PI = M_PI;

        public:
        double invert_func(double r) { return pow(tan(PI * r / 2.0), 1.0 / 3.0); }

        struct Trajectory {
                std::vector<double> x_coords;
                std::vector<double> y_coords;
        };

        Trajectory simulate_with_trajectory(int N, std::mt19937& rng) {
                Trajectory traj;
                traj.x_coords.reserve(N + 1);
                traj.y_coords.reserve(N + 1);

                double x = 0.0, y = 0.0;
                traj.x_coords.push_back(x);
                traj.y_coords.push_back(y);

                std::uniform_real_distribution<> dis(0.0, 1.0);
                std::uniform_int_distribution<> dir_dis(0, 3);

                for (int step = 0; step < N; step++) {
                        double r = dis(rng);
                        double length = invert_func(r);
                        int direction = dir_dis(rng);

                        switch (direction) {
                                case 0:
                                y += length;
                                break;
                                case 1:
                                y -= length;
                                break;
                                case 2:
                                x -= length;
                                break;
                                case 3:
                                x += length;
                                break;
                        }

                        traj.x_coords.push_back(x);
```

```
                        traj.y_coords.push_back(y);
                }

                return traj;
        }

        std::vector<Trajectory> simulate_parallel_with_trajectories(int M, int N, int num_threads = 4) {
                if (num_threads > 0) {
                        omp_set_num_threads(num_threads);
                }

                std::vector<Trajectory> results(M);

                #pragma omp parallel for
                for (int i = 0; i < M; i++) {
                        std::mt19937 thread_rng(std::random_device{}() + omp_get_thread_num() * 12345);
                        results[i] = simulate_with_trajectory(N, thread_rng);
                }

                return results;
        }

        Statistics compute_statistics_from_trajectories(const std::vector<Trajectory>& trajectories) {
                Statistics stats;
                int M = trajectories.size();

                stats.x_coords.reserve(M);
                stats.y_coords.reserve(M);
                stats.trajectories_x.reserve(M);
                stats.trajectories_y.reserve(M);

                double sum_x = 0.0, sum_y = 0.0;
                double sum_x2 = 0.0, sum_y2 = 0.0;

                for (const auto& traj : trajectories) {
                        double x_final = traj.x_coords.back();
                        double y_final = traj.y_coords.back();

                        stats.x_coords.push_back(x_final);
                        stats.y_coords.push_back(y_final);

                        stats.trajectories_x.push_back(traj.x_coords);
                        stats.trajectories_y.push_back(traj.y_coords);

                        sum_x += x_final;
                        sum_y += y_final;
                        sum_x2 += x_final * x_final;
                        sum_y2 += y_final * y_final;
                }

                stats.mean_x = sum_x / M;
                stats.mean_y = sum_y / M;
                stats.var_x = sum_x2 / M - stats.mean_x * stats.mean_x;
                stats.var_y = sum_y2 / M - stats.mean_y * stats.mean_y;
                stats.var_R = stats.var_x + stats.var_y;
                stats.mean_R = sqrt(stats.mean_x * stats.mean_x + stats.mean_y * stats.mean_y);

                return stats;
        }

        void send_to_python_and_plot(const std::vector<Trajectory>& trajectories, int M, int N,
        double elapsed_time) {
                Statistics stats = compute_statistics_from_trajectories(trajectories);

                py::array_t<double> x_coords(stats.x_coords.size());
                py::array_t<double> y_coords(stats.y_coords.size());

                auto x_buf = x_coords.request();
```

```
                auto y_buf = y_coords.request();

                double* x_ptr = static_cast<double*>(x_buf.ptr);
                double* y_ptr = static_cast<double*>(y_buf.ptr);

                for (size_t i = 0; i < stats.x_coords.size(); i++) {
                        x_ptr[i] = stats.x_coords[i];
                        y_ptr[i] = stats.y_coords[i];
                }

                py::list traj_x_list, traj_y_list;

                int num_trajectories_to_save = std::min(10, (int)trajectories.size());
                for (int i = 0; i < num_trajectories_to_save; i++) {
                        py::list traj_x, traj_y;
                        for (size_t j = 0; j < trajectories[i].x_coords.size(); j++) {
                                traj_x.append(trajectories[i].x_coords[j]);
                                traj_y.append(trajectories[i].y_coords[j]);
                        }
                        traj_x_list.append(traj_x);
                        traj_y_list.append(traj_y);
                }

                py::dict data;
                data["x_coords"] = x_coords;
                data["y_coords"] = y_coords;
                data["trajectories_x"] = traj_x_list;
                data["trajectories_y"] = traj_y_list;
                data["mean_x"] = stats.mean_x;
                data["mean_y"] = stats.mean_y;
                data["var_x"] = stats.var_x;
                data["var_y"] = stats.var_y;
                data["var_R"] = stats.var_R;
                data["mean_R"] = stats.mean_R;
                data["M"] = M;
                data["N"] = N;
                data["time"] = elapsed_time;

                try {
                        py::module_ plotter = py::module_::import("main");
                        plotter.attr("plot_simulation_results")(data);
                } catch (const py::error_already_set& e) {
                        std::cerr << "Python error: " << e.what() << std::endl;
                }
        }

        void run_with_visualization(const std::vector<int>& M_values, const std::vector<int>& N_values,
        int num_threads = 4) {
                try {
                        py::module_::import("matplotlib");
                        std::cout << "Matplotlib imported successfully\n";
                } catch (const py::error_already_set& e) {
                        std::cerr << "Failed to import matplotlib: " << e.what() << std::endl;
                        return;
                }

                for (int M : M_values) {
                        for (int N : N_values) {
                                std::cout << "M = " << M << ", N = " << N << std::flush;

                                auto start = std::chrono::high_resolution_clock::now();

                                auto trajectories = simulate_parallel_with_trajectories(M, N, num_threads);

                                auto end = std::chrono::high_resolution_clock::now();
                                double elapsed = std::chrono::duration<double>(end - start).count();

                                std::cout << ", done in " << elapsed << "s" << std::endl;

```

```
                                send_to_python_and_plot(trajectories, M, N, elapsed);
                        }
                }

                try {
                        py::module_ plotter = py::module_::import("main");
                        plotter.attr("plot_summary_plots")();
                } catch (const py::error_already_set& e) {
                        std::cerr << "Python error in summary: " << e.what() << std::endl;
                }
        }
};

int main(int argc, char* argv[]) {
        py::scoped_interpreter guard{};

        try {
                py::module_ sys = py::module_::import("sys");
                py::list path = sys.attr("path");

                path.attr("insert")(0, ".");

                std::string src_path =
                    "/home/aigul/Documents/vs_code_projects/matmod/labs/lr4/src";
                path.attr("insert")(0, src_path);

                std::cout << "Python sys.path configured\n";
        } catch (const py::error_already_set& e) {
                std::cerr << "Failed to configure Python path: " << e.what() << std::endl;
        }

        RandomWalksVisualizer simulator;

        std::vector<int> M_values = {100, 1000, 10000};
        std::vector<int> N_values;
        for (int N = 500; N <= 5000; N += 500) {
                N_values.push_back(N);
        }

        simulator.run_with_visualization(M_values, N_values, 8);

        std::cout << "\nPress Enter to exit...";
        std::cin.get();

        return 0;
}
```

main.py

```
import matplotlib

matplotlib.use("Agg")

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

current_dir = Path(__file__).parent.absolute()
project_root = current_dir.parent.parent.parent
RESULTS_DIR = project_root / "labs/lr4/out"
RESULTS_DIR.mkdir(exist_ok=True)

all_results = {}

def plot_simulation_results(data):
        M = data["M"]
        N = data["N"]
        x_coords = np.array(data["x_coords"])
        y_coords = np.array(data["y_coords"])
        trajectories_x = data["trajectories_x"]
        trajectories_y = data["trajectories_y"]
```

```
        mean_x = data["mean_x"]
        mean_y = data["mean_y"]
        var_x = data["var_x"]
        var_y = data["var_y"]
        var_R = data["var_R"]
        mean_R = data["mean_R"]

        if M not in all_results:
        all_results[M] = {}
        all_results[M][N] = {
                "x_coords": x_coords,
                "y_coords": y_coords,
                "trajectories_x": trajectories_x,
                "trajectories_y": trajectories_y,
                "mean_x": mean_x,
                "mean_y": mean_y,
                "var_x": var_x,
                "var_y": var_y,
                "var_R": var_R,
                "mean_R": mean_R,
        }

        if len(trajectories_x) > 0:
                plot_trajectory(trajectories_x[0], trajectories_y[0], M, N)
        plot_mnk_for_each_M()


def plot_mnk_for_each_M():
        for M, results in all_results.items():
        N_values = sorted(results.keys())
        var_R_values = [results[N]["var_R"] for N in N_values]

        log_N = np.log(N_values)
        log_var = np.log(var_R_values)
        nu, ln_A = coeffs[0], coeffs[1]
        A = np.exp(ln_A)

        N_fit = np.linspace(min(N_values), max(N_values), 100)
        var_R_fit = A * N_fit**nu

        fig, ax = plt.subplots(figsize=(8, 6))
        ax.loglog(
                N_values,
                var_R_values,
                "o-",
                label="<\Delta R^2>",
                markersize=6,
                linewidth=2,
                )
                ax.loglog(
                N_fit, var_R_fit, "r--", label=f"MNK: <$\Delta$R>={A:.3f}*N^{nu:.3f}", linewidth=2
        )

        ax.set_xlabel("N", fontsize=11)
           ax.set_ylabel("<delta R^2>", fontsize=11)
        ax.legend(fontsize=10)

        plt.tight_layout()
        filename = RESULTS_DIR / f"mnk_M{M}.png"
        plt.savefig(filename, dpi=150, bbox_inches="tight")
        plt.close(fig)


def plot_trajectory(traj_x, traj_y, M, N):
        fig, ax = plt.subplots(figsize=(9, 8))

        ax.plot(traj_x, traj_y, "b-", linewidth=1.5, alpha=0.8)
        ax.plot(traj_x[0], traj_y[0], "go", markersize=6, label="Start")
        ax.plot(traj_x[-1], traj_y[-1], "ro", markersize=6, label="Finish")

        ax.set_xlabel("x")
        ax.set_ylabel("y")
```

```python
        ax.set_title(f"First trajectory (M={M}, N={N})")
        ax.grid(True, alpha=0.3)
        ax.axis("equal")
        ax.legend(loc="upper right")

        plt.tight_layout()

        filename = RESULTS_DIR / f"trajectory_M{M}_N{N}.png"
        plt.savefig(filename, dpi=150)

        plt.close(fig)


def plot_summary_plots():
        fig, ax = plt.subplots(figsize=(10, 6))
        for M, results in all_results.items():
                N_values = sorted(results.keys())
                var_R_values = [results[N]["var_R"] for N in N_values]
                ax.loglog(N_values, var_R_values, "o-", label=f"M={M}", markersize=6)

                if len(N_values) > 1:
                        log_N = np.log(N_values)
                        log_var = np.log(var_R_values)
                        coeffs = np.polyfit(log_N, log_var, 1)
                        nu = coeffs[0]
                        A = np.exp(coeffs[1])
                        theory = A * np.array(N_values) ** nu
                        ax.loglog(N_values, theory, "--", alpha=0.5, label=f"M={M} (nu={nu:.3f})")


        ax.set_xlabel("N")
        ax.set_ylabel("<delta R^2>")
        ax.grid(True, alpha=0.3, which="both")
        ax.legend(fontsize=10)

        filename = RESULTS_DIR / "var_R_vs_N.png"
        plt.savefig(filename, dpi=150, bbox_inches="tight")
        plt.close(fig)


        for M, results in all_results.items():
                fig, axes = plt.subplots(1, 2, figsize=(12, 5))

                N_values = sorted(results.keys())
                mean_R_values = [results[N]["mean_R"] for N in N_values]
                var_R_values = [results[N]["var_R"] for N in N_values]

                axes[0].plot(
                N_values, mean_R_values, "o-", label="<R>", markersize=4, color="blue"
                )
                axes[0].set_xlabel("N")
                axes[0].set_ylabel("<R>")
                axes[0].grid(True, alpha=0.3)
                axes[0].legend()


                axes[1].plot(
                        N_values, var_R_values, "s-", label="ro^2(R)", markersize=4, color="red"
                )
                axes[1].set_xlabel("N")
                axes[1].set_ylabel("Variance")
                axes[1].set_title(f"Variance (M={M})")
                axes[1].grid(True, alpha=0.3)
                axes[1].legend()

                plt.tight_layout()
                filename = RESULTS_DIR / f"statistics_M{M}.png"
                plt.savefig(filename, dpi=150, bbox_inches="tight")
                plt.close(fig)

        plot_histograms_for_each_M()
```

```python


def plot_histograms_for_each_M():
    for M, results in all_results.items():
        fig, axes = plt.subplots(1, 2, figsize=(14, 6))
        fig.suptitle(f"Frequency M={M}", fontsize=14)

        all_x_coords = []
        all_y_coords = []

        for N, stats in sorted(results.items()):
        all_x_coords.extend(stats["x_coords"])
        all_y_coords.extend(stats["y_coords"])

        all_x_coords = np.array(all_x_coords)
        all_y_coords = np.array(all_y_coords)

        total_mean_x = np.mean(all_x_coords)
        total_std_x = np.std(all_x_coords)
        total_mean_y = np.mean(all_y_coords)
        total_std_y = np.std(all_y_coords)

        ax1 = axes[0]
        n_x, bins_x, patches_x = ax1.hist(
                all_x_coords,
                bins=50,
                alpha=0.7,
                color="blue",
                edgecolor="black",
                linewidth=1,
        )
        ax1.set_xlabel("x")
        ax1.set_ylabel("Frequency")
        ax1.set_title("Distribution by x")
        ax1.grid(True, alpha=0.3)

        ax2 = axes[1]
        n_y, bins_y, patches_y = ax2.hist(
                all_y_coords,
                bins=50,
                alpha=0.7,
                color="green",
                edgecolor="black",
                linewidth=1,
        )
        ax2.set_xlabel("y")
        ax2.set_ylabel("Frequency")
        ax2.set_title("Distribution by y")
        ax2.grid(True, alpha=0.3)

        plt.tight_layout()
        filename = RESULTS_DIR / f"histograms_M{M}.png"
        plt.savefig(filename, dpi=150, bbox_inches="tight")
        print(
        f"M={M}: mean_x={total_mean_x:.4f}, std_x={total_std_x:.4f},
            mean_y={total_mean_y:.4f}, std_y={total_std_y:.4f}"
        )
        plt.close(fig)
```